todd_5530_m6

AUTHOR
Reagan Todd

```
R.version.string
```

```
[1] "R version 4.5.2 (2025-10-31)"
```

```
Sys.Date()
```

```
[1] "2026-03-27"
```

```
library(tidyverse)
library(DBI)
library(RSQLite)
library(readr)
```

This is a demonstration of how to join tables in SQL created

2026-03-27 by Reagan Todd. The copyright is for educational uses only.

Q1. Use the labevents table in the MIMICIII to select glucose levels (WHERE itemid = 50809). Use CASE WHEN to categorize glucose levels, if valuenum > 140 categorize as 'High' otherwise categorize as 'Normal_or_Low'.

First we need to add labevents to our MIMICIII database:

```
con <- dbConnect(SQLite(), "mimic_demo.db")

labevents <- read_csv(
  "~/Documents/Spring 26/MEDB-5530/data/mimic-iii-demo/LABEVENTS.csv"
)
```

```
Rows: 76074 Columns: 9
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr  (3): value, valueuom, flag
dbl  (5): row_id, subject_id, hadm_id, itemid, valuenum
dttm (1): charttime
```

```
ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
dbWriteTable(con, "labevents", labevents, overwrite = TRUE)
```

Now we can categorize the glucose

```
cox <- dbConnect(SQLite(), "mimic_demo.db")
labevents <- dbGetQuery(conn=cox, "
SELECT
  subject_id,
  valuenum AS glucose,
  CASE
    WHEN valuenum > 140 THEN 'High'
    ELSE 'Normal_or_low'
  END AS glucose_category
FROM labevents
WHERE itemid = 50809 -- Glucose
AND valuenum IS NOT NULL
LIMIT 10;")

labevents
```

```
   subject_id glucose glucose_category
1       10006      77    Normal_or_low
2       10019     146             High
3       10019     158             High
4       10019     150             High
5       10019     159             High
6       10019      80    Normal_or_low
7       10019     287             High
8       10027     168             High
9       10027     148             High
10      10027     166             High
```

```
dbDisconnect(cox)
```

Interpretation of output

We can now see that we have the glucose value and a glucose category. In just the first 10 rows, we can see that we only have two values under 140 those are categories “Normal_or_low” and we have eight of “High” category. Just from the first 10 rows we can see that the case-when conditional formatting worked.

Interpretation of code

Here we see subject ID, glucose level and the glucose category. We categorized the glucose into category using the case where where we stated that if the valuenum (glucose) was >140 then show in

category high show in the category of normal or low. A good reference to understanding case-when and else can be found here.

Q2. Use INNER JOIN in MIMIC-III to join patients table and and Admissions table. Get age, gender and admission details for patients. (limit 10 rows)

First I want to ensure that my data is loaded correctly for patients

and admissions.

```
con <- dbConnect(SQLite(), "mimic_demo.db")

patients <- read_csv(
  "~/Documents/Spring 26/MEDB-5530/data/mimic-iii-demo/PATIENTS.csv"
)
```

```
Rows: 100 Columns: 8
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr  (1): gender
dbl  (3): row_id, subject_id, expire_flag
dttm (4): dob, dod, dod_hosp, dod_ssn

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
dbWriteTable(con, "patients", patients, overwrite = TRUE)

admissions <- read_csv(
  "~/Documents/Spring 26/MEDB-5530/data/mimic-iii-demo/ADMISSIONS.csv"
)
```

```
Rows: 129 Columns: 19
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr  (9): admission_type, admission_location, discharge_location, insurance,...
dbl  (5): row_id, subject_id, hadm_id, hospital_expire_flag, has_chartevents...
dttm (5): admittime, dischtime, deathtime, edregtime, edouttime

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
dbWriteTable(con, "admissions", admissions, overwrite = TRUE)
```

Now we can inner join

Note this will find the age at admission due to it not being provided.

```
cox <- dbConnect(SQLite(), "mimic_demo.db")
patients_admissions <- dbGetQuery(conn=cox, "
SELECT
  p.subject_id,
  p.gender,
  a.hadm_id,
  a.admittime,
  a.dischtime,
  p.dob,
  DATE(p.dob, 'unixepoch') AS birth_date,
  DATE(a.admittime, 'unixepoch') AS admit_date,
  CAST(strftime('%Y', a.admittime, 'unixepoch') AS INTEGER) AS admit_year,
  ROUND(
    (julianday(a.admittime, 'unixepoch') - julianday(p.dob, 'unixepoch')) / 365.25,
    1
  ) AS age_at_admit
FROM patients p
INNER JOIN admissions a
  ON p.subject_id = a.subject_id
LIMIT 10; ")


patients_admissions
```

```
   subject_id gender hadm_id  admittime  dischtime         dob birth_date
1       10006      F  142345 6147695340 6148458900  3918585600 2094-03-05
2       10011      F  105331 4942420320 4943617140  3800304000 2090-06-05
3       10013      F  165520 4915294560 4915523580  2167084800 2038-09-03
4       10017      F  199207 5661335940 5662032120  3336249600 2075-09-21
5       10019      M  177759 6102074580 6102129600  4558896000 2114-06-20
6       10026      F  103770 7112129940 7112749500 -2355004800 1895-05-17
7       10027      F  199395 6959286900 6960348000  4356028800 2108-01-15
8       10029      M  132349 5355975480 5356852140  2880316800 2061-04-10
9       10032      M  140372 5309524320 5310628500  2532124800 2050-03-29
10      10033      F  157235 5141501160 5141805300  2565648000 2051-04-21
   admit_date admit_year age_at_admit
1  2164-10-23       2164         70.6
2  2126-08-14       2126         36.2
3  2125-10-04       2125         87.1
4  2149-05-26       2149         73.7
5  2163-05-14       2163         48.9
6  2195-05-17       2195        300.0
7  2190-07-13       2190         82.5
8  2139-09-22       2139         78.4
```

```
9  2138-04-02       2138         88.0
10 2132-12-05       2132         81.6
```

```
dbDisconnect(cox)
```

Interpretation of output

Here we now have an age_at_admit column. This shows the age at which a person was when they were admitted in years. For the first 10 rows we can see most participants were in their 70's or 80's with a few under 50. The age of 300 represents an individual who was older than 89 years of age at admission.

Interpretation of code

Date is the SQLite version of the TO_DATE as shown in lecture. It returns the date as text format of YYYY-MM-DD. CAST allowed us to change the format of admit_year from any other format into an integer.
julianday this is the julian day, the fractional number since a reference day. unixepoch returns a unix timestamp which is seen in some of the records where we get the number of seconds since a predetermined date. Lastly, the strftime() function for '%Y' returns just the year, since we wanted to calculate age we can use the year to determine that.

Q3. Use the MIMICIII schema and write a SQL query that will list all the table names.

```
cox <- dbConnect(SQLite(), "mimic_demo.db")
table_names <- dbGetQuery(conn=cox, "
SELECT name AS table_name
FROM sqlite_master
WHERE type='table'
ORDER BY name; ")

table_names
```

```
  table_name
1 admissions
2 drug_codes
3  labevents
4   patients
```

```
dbDisconnect(cox)
```

Interpretation of output

As you can see, we have only used the three tables as shown above. This version shows just the tables that I have created in the past 6 modules using the MIMICIII database.

Using the mimiciii.db provided, we can see all of the tables in mimiciii.

```
cox <- dbConnect(SQLite(), "~/Documents/Spring 26/MEDB-5530/data/mimiciii.db")

tables <- dbGetQuery(conn=cox, "
  SELECT name AS table_name
  FROM sqlite_master
  WHERE type='table'
  ORDER BY name;
")

tables
```

```
           table_name
1          ADMISSIONS
2             CALLOUT
3          CAREGIVERS
4         CHARTEVENTS
5           CPTEVENTS
6      DATETIMEEVENTS
7       DIAGNOSES_ICD
8            DRGCODES
9               D_CPT
10    D_ICD_DIAGNOSES
11   D_ICD_PROCEDURES
12            D_ITEMS
13         D_LABITEMS
14           ICUSTAYS
15     INPUTEVENTS_CV
16     INPUTEVENTS_MV
17          LABEVENTS
18 MICROBIOLOGYEVENTS
19         NOTEEVENTS
20       OUTPUTEVENTS
21           PATIENTS
22      PRESCRIPTIONS
23 PROCEDUREEVENTS_MV
24     PROCEDURES_ICD
25           SERVICES
26          TRANSFERS
```

```
dbDisconnect(cox)
```

Q4. Use the MIMICIII schema and write a SQL query that will list all the field names in the admissions table.

```
cox <- dbConnect(SQLite(), "mimic_demo.db")
table_fields <- dbGetQuery(conn=cox, "
PRAGMA table_info('admissions'); ")

table_fields
```

```
   cid                 name type notnull dflt_value pk
1    0               row_id REAL       0         NA  0
2    1           subject_id REAL       0         NA  0
3    2              hadm_id REAL       0         NA  0
4    3            admittime REAL       0         NA  0
5    4            dischtime REAL       0         NA  0
6    5            deathtime REAL       0         NA  0
7    6       admission_type TEXT       0         NA  0
8    7   admission_location TEXT       0         NA  0
9    8   discharge_location TEXT       0         NA  0
10   9            insurance TEXT       0         NA  0
11  10             language TEXT       0         NA  0
12  11             religion TEXT       0         NA  0
13  12       marital_status TEXT       0         NA  0
14  13            ethnicity TEXT       0         NA  0
15  14            edregtime REAL       0         NA  0
16  15            edouttime REAL       0         NA  0
17  16            diagnosis TEXT       0         NA  0
18  17 hospital_expire_flag REAL       0         NA  0
19  18 has_chartevents_data REAL       0         NA  0
```

```
dbDisconnect(cox)
```

Interpretation of output

Now we see the 19 columns as fields from the admissions data. This lists all of the columns. This is very similar to the R version of "colnames()".

Interpretation of code

Here I used PRAGMA_table_info this allows us to return a single row for each column name. This is specific to SQLite and is comparable to the function information_schema.columns shown in lecture. If we just want to see the list of column names, use the syntax below.

```
cox <- dbConnect(SQLite(), "mimic_demo.db")
table_field <- dbGetQuery(conn=cox, "
SELECT name
FROM pragma_table_info('admissions'); ")

table_field
```

```
                   name
1                row_id
```

```
2            subject_id
3               hadm_id
4             admittime
5             dischtime
6             deathtime
7        admission_type
8    admission_location
9    discharge_location
10            insurance
11             language
12             religion
13       marital_status
14            ethnicity
15            edregtime
16            edouttime
17            diagnosis
18 hospital_expire_flag
19 has_chartevents_data
```

```
dbDisconnect(cox)
```